

1. Глоссарий (Единый язык)

Чтобы исключить двойную трактовку, фиксируем термины:

- **Кампания** — проект по сбору средств, имеющий целевую сумму и дедлайн.
 - **Создатель (Админ кампании)** — пользователь, инициировавший кампанию.
 - **Участник (Вкладчик)** — пользователь, внесший средства в кампанию.
 - **Виртуальные средства** — криптовалюта или токены (например, ETH или ERC-20), используемые в сети.
 - **Дивиденды (Профит)** — дополнительная сумма, которую Создатель вносит в смарт-контракт после закрытия сбора для распределения среди Участников.
 - **Смарт-контракт (СК)** — код в блокчейне, хранящий средства и выполняющий логику распределения.
-

2. Основная доменная модель (Сущности)

Поскольку архитектура монолитная (Java/Spring Boot), модель будет реляционной (PostgreSQL/MySQL), но с привязкой к блокчейну.

1. User (Пользователь)

- `id` (UUID)
- `email`, `passwordHash` (для стандартной авторизации)
- `walletAddress` (адрес криптокошелька, **обязательно** для взаимодействия с Web3j)
- `role` (USER, ADMIN)

2. Campaign (Кампания)

- `id` (UUID)
- `creatorId` (ссылка на User)
- `title`, `description`
- `targetAmount` (целевая сумма в wei/smallest unit)
- `collectedAmount` (кэшированное значение из БД, синхронизируется с блокчейном)
- `status` (ENUM: DRAFT, OPEN, CLOSED, FAILED)
- `deadline` (timestamp)
- `smartContractAddress` (адрес развернутого СК)
- `transactionHashCreation` (хэш транзакции создания СК)

3. Contribution (Взнос)

- `id` (UUID)

- `campaignId`
- `userId`
- `amount`
- `blockchainTxHash` (хэш транзакции в блокчейне)
- `timestamp`

4. **DividendPayout (Выплата дивидендов)**

- `id` (UUID)
 - `campaignId`
 - `creatorId`
 - `profitAmount` (сумма профита, внесенная создателем)
 - `blockchainTxHash`
 - `timestamp`
-

3. Юзкейсы (Use Cases)

UC-01: Регистрация и привязка кошелька

- **Актор:** Гость
- **Сценарий:** Пользователь регистрируется по email/паролю. **Важно:** на этапе регистрации или сразу после входа система требует подписать сообщение (Sign Message) своим криптокошельком (MetaMask и т.д.), чтобы привязать `walletAddress` к аккаунту в БД. Без этого он не сможет донатить.

UC-02: Просмотр публичной информации

- **Актор:** Гость / Пользователь
- **Сценарий:**
 - На Главной странице отображаются топ-N кампаний (сортировка по `collectedAmount` или дате).
 - Поиск по названию/описанию (без фильтров по категориям).
 - Страница кампании показывает все поля из доменной модели. Статусы:
 - `OPEN` (идет сбор)
 - `CLOSED` (цель достигнута, сбор закрыт)
 - `FAILED` (дедлайн истек, цель не достигнута — *нужно уточнить у заказчика, что происходит с деньгами, см. раздел Риски*).

UC-03: Создание кампании

- **Актор:** Авторизованный пользователь
- **Сценарий:** Пользователь заполняет форму (Название, Описание, Целевая сумма, Дедлайн).
- **Техническое действие:** Бэкенд с помощью **Web3j** генерирует и деплоит в сеть новый Смарт-контракт, передавая в него адрес создателя и целевую сумму. Адрес СК сохраняется в БД. Статус кампании становится `OPEN`.

UC-04: Внесение средств (Донат)

- **Актор:** Авторизованный пользователь
- **Сценарий:** На странице `OPEN` кампании пользователь нажимает "Внести средства". Фронтенд (через `Web3.js/Ethers.js`) вызывает функцию `contribute()` в СК.
- **Техническое действие:**
 1. Фронтенд отправляет транзакцию.
 2. Бэкенд (Java) слушает события блокчейна (Event Listener через `Web3j`). Когда транзакция подтверждается, бэкенд обновляет `collectedAmount` в БД.
 3. Если `collectedAmount >= targetAmount`, бэкенд меняет статус на `CLOSED` и блокирует прием новых средств на уровне СК.

UC-05: Личный кабинет

- **Актор:** Авторизованный пользователь
- **Сценарий:** Вкладки:
 - "Мои кампании" (где он Создатель).
 - "Мои вклады" (где он Участник, с указанием суммы и текущего статуса выплат).
 - "Баланс" (общая сумма ожидаемых дивидендов).

UC-06: Распределение дивидендов (Ключевая логика)

- **Актор:** Создатель кампании (со статусом `CLOSED`)
 - **Сценарий:** Создатель вводит сумму профита. Нажимает "Распределить".
 - **Техническое действие:** Фронтенд вызывает функцию `distributeDividends(profitAmount)` в СК.
 - **Важное уточнение для бэкенда:** Бэкенд **НЕ** считает доли каждого участника. Вся математика (пропорциональное распределение) происходит **внутри Смарт-контракта**. Бэкенд только фиксирует факт вызова функции и хэш транзакции. Участники позже смогут вызвать функцию `withdraw()` в СК, чтобы забрать свои дивиденды.
-

4. Технические спецификации и ограничения (Для разработчиков)

1. **Архитектура:** Монолит (Spring Boot). Фронтенд (HTML/CSS/JS) может быть либо на Thymeleaf, либо SPA, общающаяся с бэком по REST API.
 2. **Интеграция с блокчейном (Web3j):**
 - Бэкенд не хранит приватные ключи пользователей. Все транзакции (кроме деплоя, если деплоит бэкенд) подписываются на фронтенде через Web3-провайдер (MetaMask).
 - **Синхронизация состояния:** БД выступает как кэш для UI. Бэкенд должен использовать `Web3j.subscribeToPendingTransactions` или парсить логи блоков, чтобы обновлять статусы кампаний в БД, как только они меняются в блокчейне.
 3. **Газ (Комиссия сети):**
 - При донате газ платит Участник.
 - При распределении дивидендов газ платит Создатель (так как он инициирует транзакцию). *Это нужно явно указать в UI, чтобы Создатель не удивился списанию комиссии.*
-

5. Условия приёмки (Acceptance Criteria)

Задача считается выполненной, если пройдены следующие проверки:

1. **Блокировка сбора:** Если сумма сборов достигает `targetAmount`, статус кампании меняется на `CLOSED`. Попытка внести средства после этого отклоняется Смарт-контрактом (реверт транзакции), а на фронте кнопка "Внести" становится неактивной.
2. **Пропорциональность:** Если Участник А внес 10%, а Участник Б 90%, то при распределении профита в 100 монет, СК автоматически начисляет 10 монет А и 90 монет Б. Ошибки округления (`wei`) допустимы в пределах 1 `wei`.
3. **Прозрачность:** На странице кампании в блоке "Транзакции" отображаются хэши всех транзакций (взносы, выплаты), ведущие на блокчейн-эксплорер (например, Etherscan).
4. **Отказоустойчивость:** Если фронтенд "потерял" связь с блокчейном, пользователь видит актуальные данные из БД (кэша), а при восстановлении связи данные подтягиваются.